The three-lamp interlock

Lesson 023 — ask, admit, light exactly one

A deliberate failure makes every load lamp go dark; then the circuit recovers.

Time	about 35 minutes	**Board**	Arduino Mega 2560
Payoffs	chase, fail-closed scene, recovery	**Energy**	E1, USB only
Physical parts	four LEDs, three 220 Ω, one 1 kΩ	**Loads**	names only

This is a three-lamp policy machine. “Fan,” “pump,” and “heater” are names for three ordinary LEDs. The sketch never connects or controls a relay, driver, motor, pump, heater, external supply, or mains circuit. D13 blinks before each request; D40, D38, and D41 show the one admitted choice; D12 shows the deliberate fault.

Hard boundary. Keep every relay module, transistor board, motor, external supply, and energetic load off the table. An LED demonstrates software intent, not isolation or safe load power. Unplug USB before moving a wire. Stop for heat, smell, smoke, repeated USB resets, two load lamps glowing together, or any disagreement between the plate and the circuit.

Adventure map

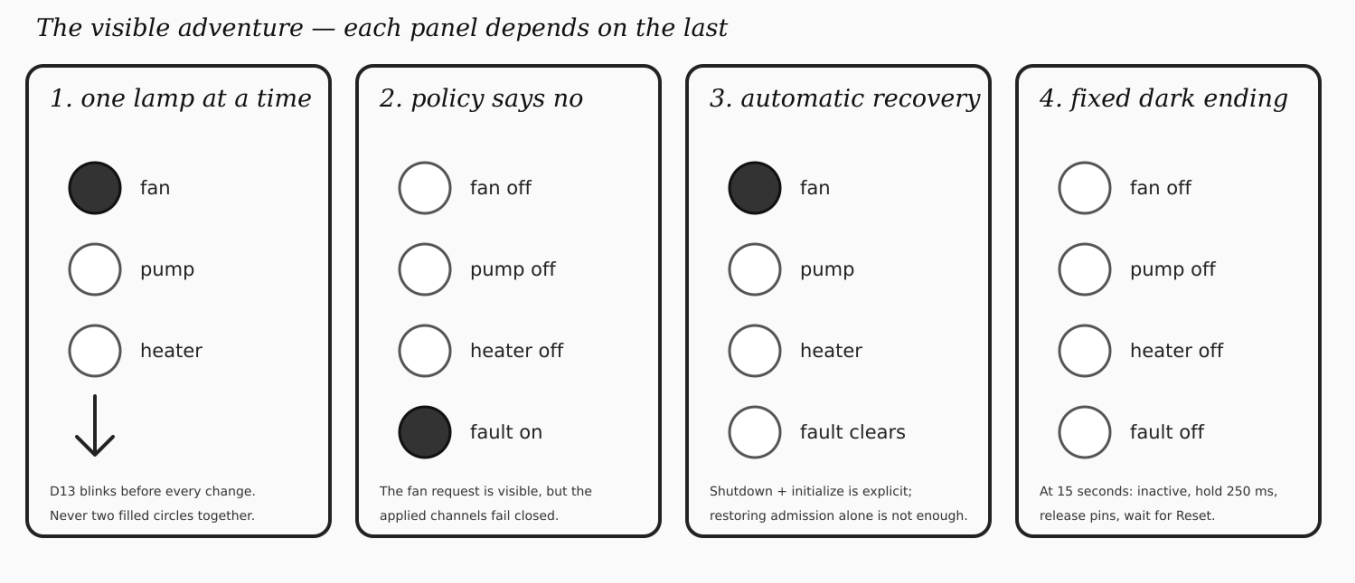

Plate 1: four dependent visible wins. First the three applied lamps chase. Next a rejected fan request makes all three dark while D12 stays on for two seconds. Explicit shutdown and initialization clear that sticky software fault, so the chase succeeds again. At 15 seconds every lamp goes dark, stays inactive for 250 ms, releases its pin, and waits for Reset. No input can extend that deadline.

Stage 1: build one literal breadboard

Gather a Mega 2560, USB cable, breadboard, four ordinary LEDs, three 220 Ω resistors, one 1 kΩ resistor, and jumpers. Identify each LED’s anode and cathode from the actual part; do not infer polarity from color. The built-in D13 LED needs no wire.

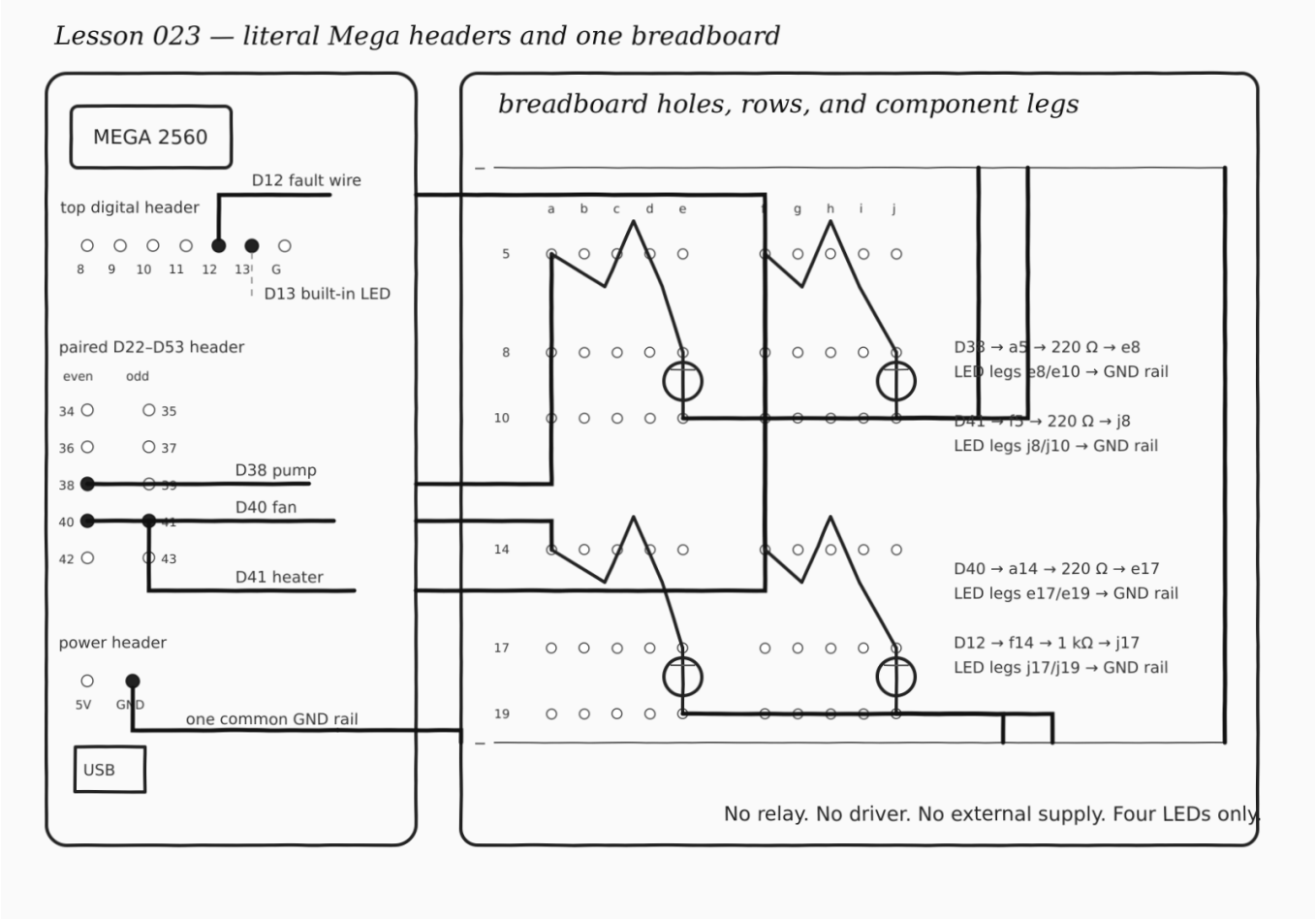

Plate 2: actual header pairs, breadboard rows, and component legs. The paired Mega header puts even D38 and D40 in one physical row and odd D41 beside D40. D12 is on the top digital header. Breadboard holes a–e in one numbered row are connected; f–j form a separate group across the center gap. The two negative rails are joined at the far end and return to one Mega GND.

Lamp	Mega pin	Exact breadboard path
Pump	D38	jumper to a5; 220 Ω a5–e8; LED e8 anode/e10 cathode; row 10 to negative rail
Fan	D40	jumper to a14; 220 Ω a14–e17; LED e17 anode/e19 cathode; row 19 to negative rail
Heater	D41	jumper to f5; 220 Ω f5–j8; LED j8 anode/j10 cathode; row 10 to negative rail
Fault	D12	jumper to f14; 1 kΩ f14–j17; LED j17 anode/j19 cathode; row 19 to negative rail
Return	GND	Mega GND to negative rail; bridge the two negative rails

Build the pump branch first. Compare a5, e8, e10, and the negative rail with Plate 2. Add fan, heater, then fault one branch at a time. This order keeps a mistake local and makes the finished layout easy to compare hole by hole.

Stage 2: watch mutual exclusion

Restore USB and upload the canonical `Lesson023InertLoadInterlock` sketch. Within a third of a second D13 blinks and the fan lamp on D40 lights. The visible sequence is:

D40 fan → D38 pump → D41 heater → all off.

Each change begins with a short D13 pulse. One applied lamp turns off before the next turns on. If a lamp is missing, unplug and inspect only its named pin, resistor holes, LED legs, and ground return. If two glow together, unplug immediately and compare every occupied hole with Plate 2.

This win depends on the complete path:

request pulse → selection → admission → panel → one LED.

Serial output is not needed.

Stage 3: enjoy the fail-closed scene

After the first all-off moment, the sketch deliberately revokes its *software* command admission and asks for fan again. D13 still blinks, but D40, D38, and D41 remain dark. D12 lights for about two seconds.

That disagreement is the point: a request happened, policy refused to apply it, and a distinct lamp made the fault visible. The scene supports a fail-closed software decision in this inert circuit. It does not prove galvanic isolation, relay-contact position, stored-energy removal, or safe control of a real load.

Stage 4: recovery earns the second chase

Restoring admission alone cannot erase the sticky fault. The sketch explicitly shuts down the panel and interlock, restores admission, initializes both, and clears D12. Only then does the fan–pump–heater chase run a second time.

```
panel.shutdown ();
interlock.shutdown ();
admission.admit ();

if (interlock.initialize ().ok () &&
    panel.initialize ().ok ())
{
    faultEvidence.off ();
}
```

The second chase is therefore a dependent reward: it appears only if acquisition, the first chase, deliberate rejection, all-off application, sticky-fault handling, and explicit lifecycle recovery all worked.

Stage 5: discover the fixed ending

At 15 seconds, regardless of the active stage, the sketch revokes admission, selects every simulated output inactive, turns D13 and D12 off, holds the observable inactive state for 250 ms, releases the indicator resources, and waits. Press Reset to play again.

What you see	What it supports
D13 then exactly one load lamp	a request preceded one admitted inert intent
D13, three dark load lamps, D12 on	a recorded request failed closed visibly
D12 clears, chase repeats	explicit lifecycle recovery completed
all lamps dark at 15 seconds	fixed inactive hold and shutdown began

Darkness is circuit-native evidence. Building, uploading, and watching this story still does not claim separately recorded physical verification.

Under the hood

`InertLoadInterlock` owns no pin. It records requested and active intent separately and admits at most one active value. `InertLoadPanel` borrows three `IndicatorPump` objects and renders the active value break-before-make. Construction is inert; `initialize()` acquires resources; `shutdown()` is idempotent and selects inactive before release.

The automated repository suite stays behind the scenes. It exhausts all selection transitions, invalid values, endpoint acquisition failures, old-off and new-on failures, admission revocation, sticky faults, explicit recovery, repeated shutdown, destruction, and deterministic replay. Its core trace property is

$$[\text{fan}(t)] + [\text{pump}(t)] + [\text{heater}(t)] \leq 1.$$

Those tests protect the lesson without turning the learner's visible adventure into a test procedure.

Make it yours

Change one harmless presentation detail at a time: reorder the three simulated names, make D13 pulse twice before the fault scene, shorten the ordinary chase interval, or place paper fan/pump/heater labels beside the LEDs. Keep the circuit USB-powered and resistor-limited. Never substitute a real load.

Companion material. The HTML page carries searchable pin and timing tables plus source links. This printable lesson carries the literal wiring plate, staged visible progression, and the honest inert-load boundary. Lesson 024 reuses the pump *indicator* in an inert greenhouse story.